MEMORIA DE LA PRÁCTICA:

Aplicación web para la gestión de pacientes

1. Escenario Pacientes

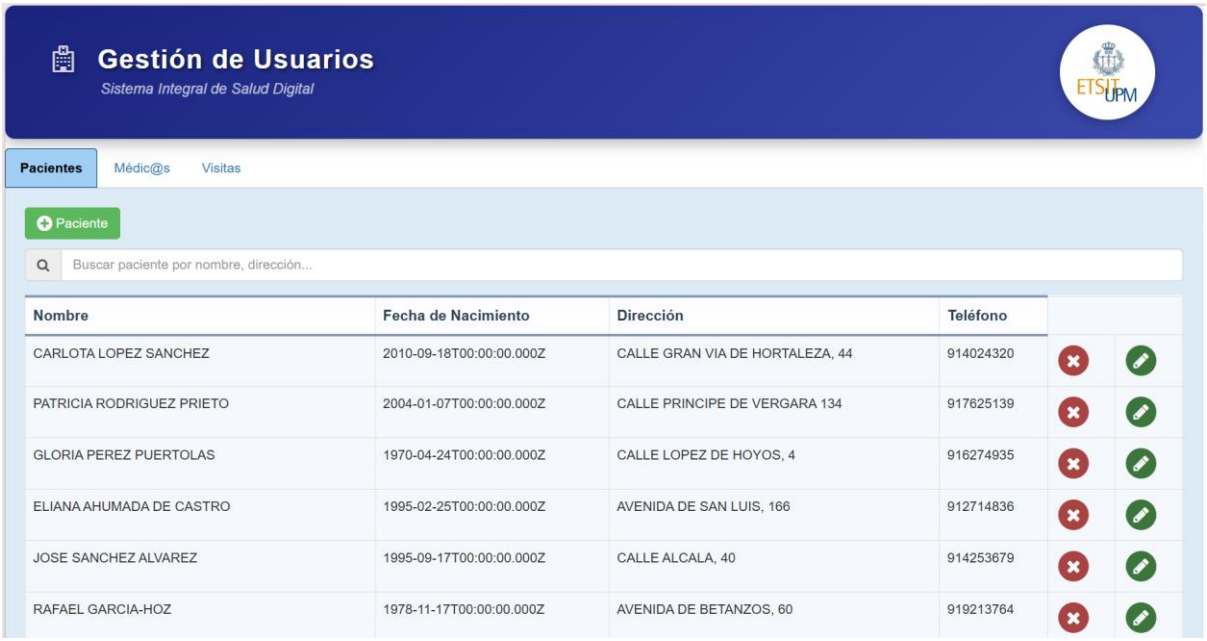


Figura 1. Escenario Pacientes.

2. Escenario Médicos

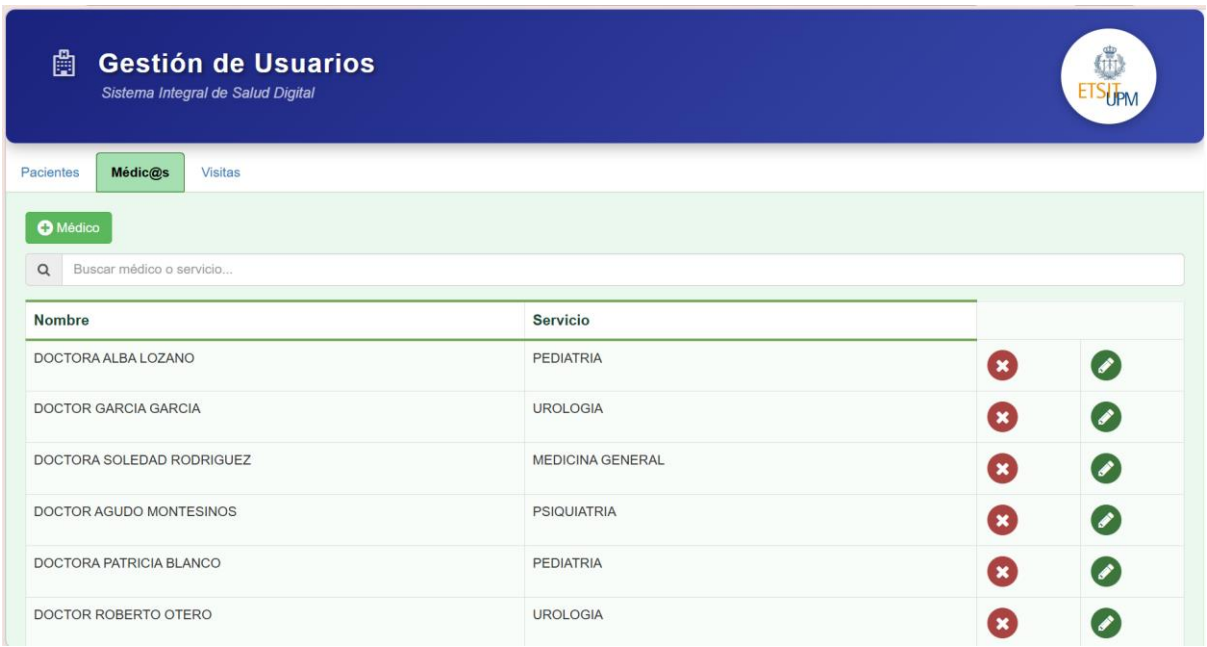


Figura 2. Escenario Médicos.

Acciones realizadas

1. **Asociar el contenido de la tabla tbmedicos al escenario de médicos**:
   Se implementó en api.js una ruta de tipo GET para consultar todos los registros de la tabla tbmedicos. En el controlador app.js, se utilizó el servicio Management.allDoctors() para recibir estos datos y almacenarlos en la variable $scope.doctors. Finalmente, en el archivo index.html, se utilizó la directiva ng-repeat para mostrar los campos MED_NOMBRE y SERVICIO en una tabla dentro de su pestaña correspondiente.

2. **Botón - formulario para añadir un nuevo médico**:
   Se habilitó el botón "Médico" vinculado a la función $scope.addNewDoctor, la cual cambia el estado de la variable $scope.newDoctor=1 para mostrar el formulario de alta mediante la directiva ng-show. Se diseñaron campos de entrada (input) vinculados mediante ng-model al objeto $scope.DoctorData.

3. **Guardar - cancelar el alta de un nuevo médico**:
   Para el guardado, se programó $scope.createNewDoctor, que envía un objeto con los datos del nuevo facultativo al servidor a través de una petición POST gestionada por el servicio Management.addDoctor(). La función de cancelación resetea el objeto DoctorData y devuelve la variable $scope.newDoctor a 0 para ocultar el formulario.

4. **Eliminar un médico**:
   Se añadió un icono de eliminación en cada fila de la tabla que invoca a $scope.deleteDoctor(id). Esta función utiliza el método Management.deleteDoc(id) para realizar una petición DELETE a la API. Tras la eliminación, el servidor devuelve la lista actualizada de médicos para refrescar la vista automáticamente.

5. **Editar los datos de un médico**:
   Se implementó la función $scope.selectedDoctor para capturar los datos del registro seleccionado y cargarlos en el formulario, activando la bandera $scope.editing=1. Al finalizar los cambios, $scope.editDoctor realiza una petición PUT mediante el servicio Management.editDoc para actualizar la información en la base de datos.

Problemas encontrados

El principal problema técnico residió en la gestión de la memoria caché del navegador. Durante el desarrollo, los cambios realizados en el código de app.js o en los estilos no se visualizaban de forma inmediata, lo que obligó a utilizar constantemente la recarga forzada (Ctrl+F5) y el modo incógnito para validar las actualizaciones. Asimismo, se detectaron errores iniciales en las consultas SQL de api.js debido a pequeñas discrepancias en los nombres de las columnas de la base de datos (como confundir MED_ID con id).

3. Escenario Visitas

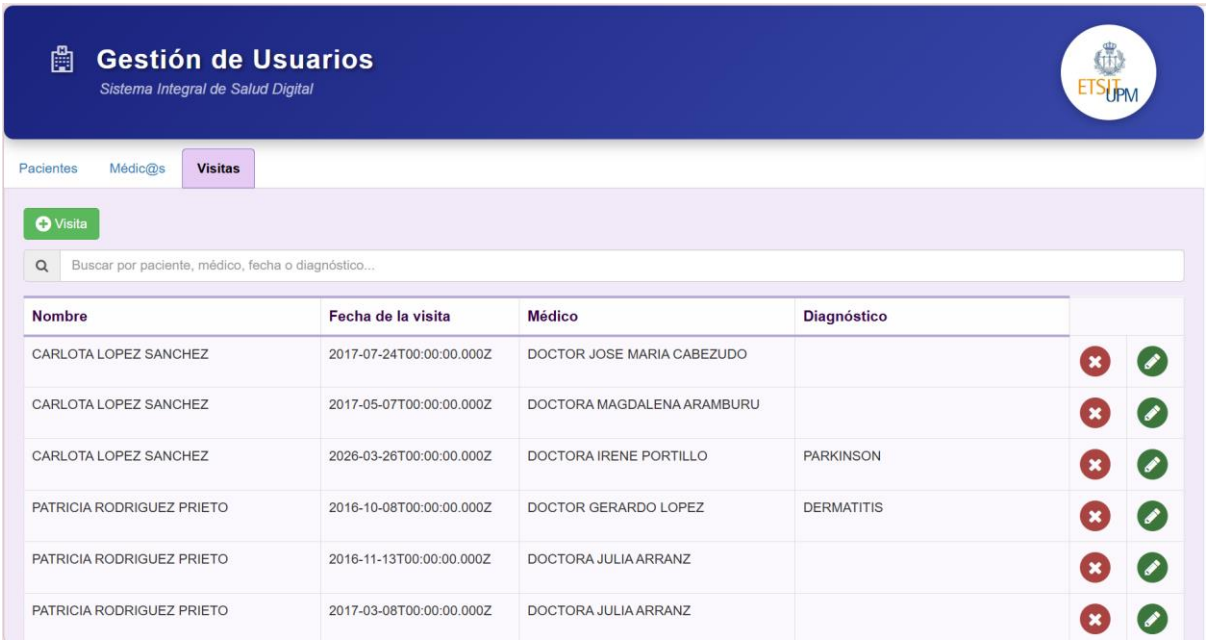


Figura 3. Escenario Visitas.

Acciones realizadas

1. **Asociar el contenido de la tabla tbvisitas al escenario de visitas**:
   Debido a que la tabla tbvisitas solo almacena identificadores numéricos (PAC_ID y MED_ID), se programó en api.js una consulta SQL compleja utilizando cláusulas JOIN para obtener los nombres reales de pacientes y médicos. Estos datos enriquecidos se cargan en $scope.visits mediante la función Management.allVisits() en el controlador.

2. **Botón - formulario para añadir una nueva visita**:
   La función $scope.addNewVisit no solo muestra el formulario, sino que dispara llamadas paralelas a Management.allPatients() y Management.allDoctors() para poblar los elementos desplegables (select) con la lista actualizada de pacientes y médicos disponibles en el sistema.

3. **Guardar - cancelar una nueva visita**:
   Se desarrolló $scope.createNewVisit, que construye un objeto con la fecha, el diagnóstico y los IDs seleccionados para enviarlos al servidor mediante una petición POST. En caso de cancelación, se limpian las listas auxiliares de pacientes y médicos para evitar duplicados en futuras aperturas del formulario.

4. **Eliminar una visita**:
   Se habilitó la funcionalidad de borrado mediante el identificador único VISITA_ID. Al ejecutar $scope.deleteVisit(id), se envía una petición DELETE a la API, la cual

elimina el registro de la tabla tbvisitas y devuelve la colección actualizada para refrescar la interfaz.

5. **Editar los datos de una visita**:
   Se implementó la función $scope.selectedVisit para realizar un mapeo de datos. Esta lógica es fundamental para que, al abrir el modo edición, el sistema localice automáticamente dentro de los arrays de pacientes y médicos aquellos cuyos IDs coinciden con los de la visita seleccionada, mostrándolos como opción predeterminada en los selectores del formulario.

Problemas encontrados

La mayor dificultad técnica en este escenario residió en la elaboración de las consultas SQL multitabla (JOINS) en api.js. Fue necesario gestionar la ambigüedad de los datos mediante el uso de alias de campo, asegurando que la unión de las tablas tbvisitas, tbpacientes y tbmedicos no generase conflictos por columnas duplicadas (como el campo nombre presente en ambas entidades de origen).

Por otro lado, se identificaron errores de asincronía en app.js al intentar ejecutar la lógica de preselección en el modo edición antes de que las promesas de carga de los arrays de pacientes y facultativos se hubieran resuelto, lo que inicialmente impedía la correcta visualización de los datos en los elementos select. Se solucionó asegurando que la asignación de datos ocurriese solo después de que las listas de médicos y pacientes se hubiesen recibido de forma exitosa (el bloque .success).

Finalmente, se detectó una incoherencia de mapeo entre el frontend y el backend durante la implementación del borrado. El botón de eliminar no realizaba ninguna acción debido a que el identificador enviado desde el HTML no coincidía exactamente con el nombre de la clave primaria esperada por la query SQL en el servidor, lo que requirió una depuración del flujo de datos para garantizar la ejecución correcta de la sentencia DELETE.

4. Modificaciones

Mejoras del aspecto

- **Distintos colores para cada escenario**:
  El objetivo es mejorar la usabilidad mediante la codificación por colores. El uso de tonos diferenciados para Pacientes (Azul), Médicos (Verde) y Visitas (Púrpura) ayuda al usuario a reducir la carga cognitiva, permitiéndole identificar el contexto de los datos de forma instantánea sin leer los encabezados. El resultado es una interfaz más intuitiva que minimiza errores de entrada en el escenario equivocado.
  - Archivo: public/views/index.html

- Implementación: Se han incluido estilos CSS personalizados (añadiendo la etiqueta <style>) dentro de la cabecera <head> para los distintos escenarios, seleccionando un tono algo más claro para el fondo que para las pestañas. Para el código de las pestañas, se ha empleado el modificador de prioridad '!important', que permite anular los estilos predefinidos por el framework Bootstrap. Además, con 'nt-child' se han definido los estilos para cada escenario.

- **Diseño de título más estético e integración de un logotipo**:
  - Archivo: public/views/index.html
  - Implementación: Uso de Flexbox y degradados CSS3 (linear-gradient) en la sección de estilos. Dentro del <body> se ha incluido un subtítulo y el path donde hemos guardado la imagen del logo de la ETSIT (una carpeta llamada img al mismo nivel que index).

- **Resaltado de las cabeceras de las tablas**:
  Esta mejora se ha implementado para fortalecer la jerarquía visual dentro de las tablas de datos de cada escenario. Al destacar la primera fila, el usuario identifica de forma instantánea los campos de información, reduciendo la fatiga visual en la lectura de registros.
  - Archivo: public/views/index.html
  - Implementación: Se ha utilizado la estructura '.tab-content .table tr:first-child td' para localizar la fila de cabecera. A diferencia de los selectores de hijo directo (>), este enfoque garantiza que el estilo se aplique independientemente de si el navegador envuelve las filas en etiquetas automáticas como <tbody>.

- **Micro-interacciones**:
  Mejoran la experiencia de usuario al proporcionar retroalimentación visual constante sobre los elementos interactivos de la interfaz de gestión.
  - Archivo: public/views/index.html
  - Implementación: Se ha incluido el efecto sombra (hover) dentro de los estilos CSS. Para los botones de acción (crear, editar y cancelar) se lleva a cabo una elevación sutil cuando situamos el cursor sobre él. Para los iconos circulares de eliminar y editar, aumenta un poco el brillo y se agranda ligeramente el icono. Por último, para las filas de la tabla, se añade una sombra al pasar el ratón.

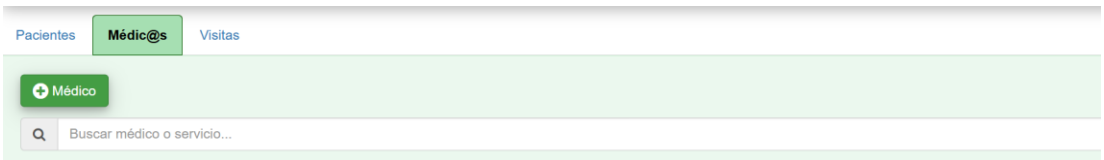


Figura 4. Elevación y efecto sombra en el botón crear.

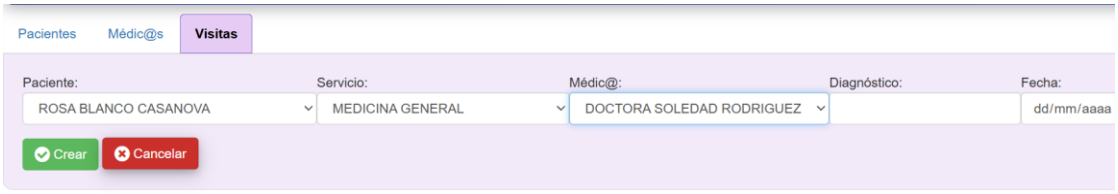


Figura 5. Elevación y efecto sombra en el botón cancelar.



Figura 6. Agrandamiento y efecto brillo en el icono de eliminar. Sombra sobre la fila seleccionada.

Mejoras de la usabilidad general

- **Avisos de confirmación para la eliminación de datos**:
  Mejoran la usabilidad por varias razones: la prevención de errores, ya que evitan que el usuario borre datos por accidente, el feedback al usuario y la seguridad de los datos, pues en un entorno de salud digital la integridad de los registros es fundamental.
  - Archivo: public/js/Dev/app.js
  - Implementación: Dentro del controlador mainControler se ha modificado el código de las funciones delete para los 3 escenarios, añadiendo un mensaje de confirmación previo y envolviendo la llamada al servicio Management dentro de un if.

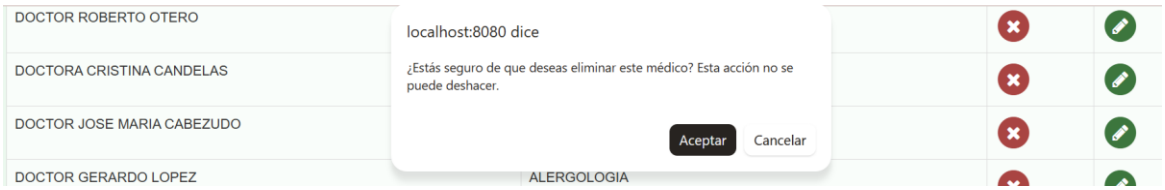


Figura 7. Aviso de confirmación para la eliminación de un médico.

- **Mensajes de éxito tras las operaciones crear, borrar y editar**:
  Proporcionan una confirmación visual inmediata de que la información se ha guardado correctamente en la base de datos. Esto reduce la incertidumbre del usuario, que no tiene que comprobar manualmente en la tabla si el dato se ha guardado o eliminado correctamente, y favorece la prevención de errores, evitando que el usuario pulse “Crear” o “Editar” varias veces por error.
  - Archivo: public/js/Dev/app.js
  - Implementación: Se ha utilizado la función alert() dentro del bloque .success de las funciones createNew, delete y edit (Patient, Doctor y Visit).

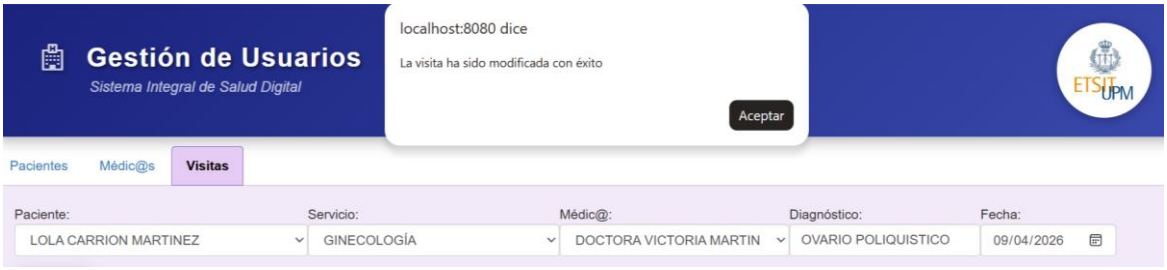


Figura 8. Mensaje de éxito tras la edición de una visita.

- **Gestión de visitas existentes mediante alertas**:
  Actualmente, si intentas borrar un médico o paciente con visitas, el servidor de MySQL lanza un error de "Foreign Key Constraint" y la aplicación se queda congelada o no responde. Vamos a crear alertas que avisen al usuario de que no se puede borrar ese paciente o médico debido a que tiene visitas asociadas. Esta implementación supone una mejora en la robustez de la página, ya que se mantiene la integridad referencial de los datos a la vez que llevamos a cabo el manejo de excepciones, mejorando la experiencia del usuario.
  - Archivos: api.js y app.js
  - Implementación: Se ha modificado tanto el servidor (api.js) para que nos avise del error, como el cliente (app.js) para que muestre la alerta al usuario. En api.js se han utilizado, dentro de las funciones .delete de Patients y Doctors, códigos de estado HTTP (409 Conflict) para comunicar errores lógicos de la base de datos entre el backend y el frontend. En app.js, en las funciones $scope.deleteDoctor y $scope.deletePatient dentro de la función '.error', se devuelve un alert si el status es igual a 409. Se ha optado por el código 409 Conflict en lugar de un error genérico para que el frontend pueda discriminar específicamente este fallo de integridad referencial y ofrecer una respuesta dirigida al usuario, evitando la incertidumbre ante errores desconocidos.



Figura 9. Aviso de la no eliminación de un paciente por tener visitas asociadas.

- **Aviso de campo incorrecto**:
  El número de teléfono del paciente a crear o editar tiene que ser de 9 dígitos.
  - Archivos: index.html y app.js
  - Implementación: En index.html se ha añadido un ng-pattern que limita el tamaño del campo teléfono a 9 dígitos En app.js, se ha añadido a las funciones createNewPatient y editPatient una variable para convertir el número a string y un

if que garantiza que, en caso de que no se hayan escrito los 9 números requeridos, se lance un aviso al usuario.

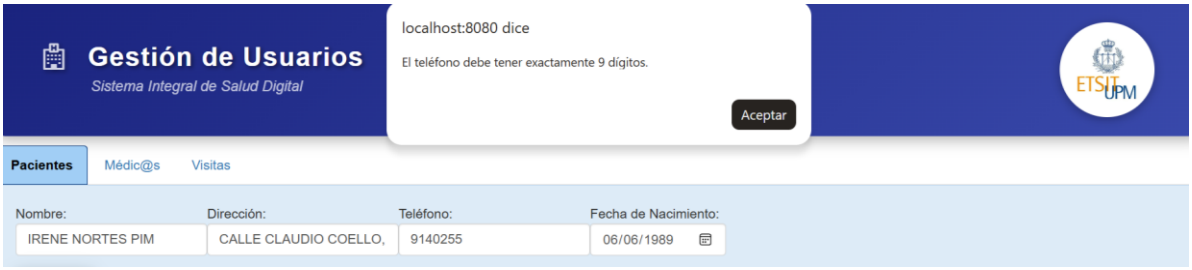


Figura 10. Aviso de número de teléfono introducido incorrecto.

Nuevas funcionalidades

- **Buscador**:
  Se han añadido buscadores en todos los escenarios para facilitar la localización de registros históricos. Esta funcionalidad reduce el trabajo de tipo administrativo del especialista, ya que permite llevar a cabo una gestión de los datos más eficiente.
  - Archivo: public/views/index.html
  - Implementación: Se ha creado un cuadro de texto para la búsqueda añadiendo un campo de entrada (input) y modificado la directiva ng-repeat de las tablas aplicando un filtro (para el escenario médicos: doctor in doctors | filter:searchDoctor).



Figura 11. Búsqueda en el escenario Pacientes por dirección.



Figura 12. Búsqueda en el escenario Visitas por nombre del médico.

- **Filtro de médicos por servicio en la creación de una visita**:
  Con esta mejora en el formulario de visitas, al elegir un servicio, el desplegable de médicos solo muestra a los doctores de ese servicio. Así se evita el error humano de asignar a un paciente un médico que no pertenece a la especialidad requerida. Además, la selección se vuelve más eficiente al no tener que visualizar todos los médicos en un mismo despegable.
  - Archivo: app.js e index.html
  - Implementación:
    En index.html, dentro del formulario de nueva visita, se ha añadido un nuevo selector para el servicio antes del selector de médicos. Además, se ha modificado el selector de médicos para que use un filtro de AngularJS. Para mejorar la usabilidad del filtro, se han empleado expresiones condicionales (operador ternario) para gestionar estados nulos en los datos. Esto permite que, al deseleccionar un servicio, el filtro se anule y el usuario pueda explorar la lista completa de facultativos sin recargar la página.
    En app.js se ha creado una nueva función $scope.getUniqueServices que extrae dinámicamente los valores únicos de la columna SERVICIO de los objetos médicos cargados en memoria. Esto se consigue mapeando la propiedad 'service' en el objeto de Angular durante la carga de médicos en el controlador, para que se genere la lista de servicios a partir de los registros de la tabla tbmedicos. Así, se consigue un sistema de filtrado dinámico basado en datos persistentes. Si se añade un médico de un servicio nuevo, el servicio aparecerá automáticamente en el filtro; si se elimina al único médico de un servicio, este desaparecerá automáticamente de la lista de visitas.
    En las funciones addNewVisit y selectedVisit, se ha reestructurado el mapeo de la lista de médicos para que cada objeto contenga la propiedad 'service'. Esto es fundamental, ya que permite al filtro de la vista comparar el servicio seleccionado

con la especialidad de cada médico, aunque dicha información no resida en la tabla de visitas de la base de datos.

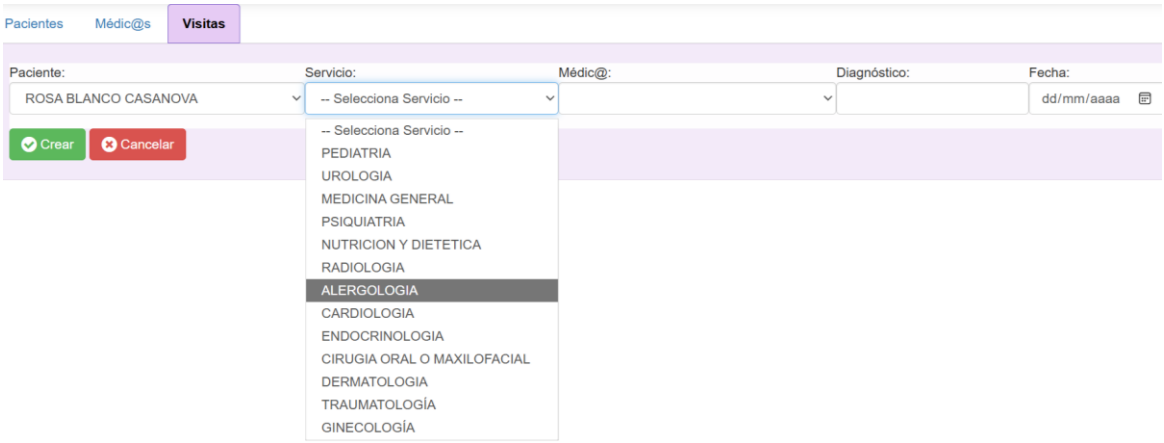


Figura 13. Selección de servicio en el escenario Visitas.

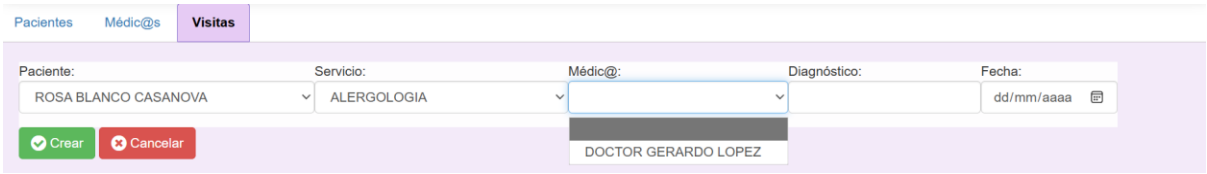


Figura 14. Selección de médico por servicio.